SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	P. MUNI KARTHIK	**Reg. No.:**	192425061
Section / Batch:	AIML	**Date:**	

Code of Conduct

I P. MUNI KARTHIK / 192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AIPowered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System

Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an ecommerce search system.

2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.
5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."** 2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.

4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.

4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	3	3	4	3	4		81
2	3	3	3	4	4		
3	4	4	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURABAN		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CASE STUDY 1: E-COMMERCE SMART SEARCH AND PRODUCT PREDICTION SYSTEM

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus: machine learning improves business, machine learning enables automation, machine learning drives innovation.

The development team uses Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

• C(machine) = 3

• C(machine learning) = 3

• C(learning improves) = 1

• C(learning enables) = 1

• C(learning drives) = 1

• C(learning improves business) = 1

The interpolation weights are:

• λ■ = 0.5 (Trigram)

• λ■ = 0.3 (Bigram)

• λ■ = 0.2 (Unigram)

The prediction probabilities after the word “learning” are:

• P(improves) = 0.33

• P(enables) = 0.33

• P(drives) = 0.33

Question 1

Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability P(learning | machine). Interpret how this probability can influence the next-word prediction of an e-commerce search system.

Answer

The Maximum Likelihood Estimate for a bigram is P(w■|w■) = C(w■,w■) / C(w■). Given C(machine, learning) = 3 and C(machine) = 3, P(learning | machine) = 3/3 = 1. Therefore, the probability is 1 or 100%. This means that in the given training corpus, every occurrence of "machine" is followed by "learning". Consequently, when a customer enters "machine", the system strongly predicts "learning" as the next word, improving autocomplete and product-search suggestions.

Question 2

A customer enters the unseen sequence "machine learning transforms". Apply the Backoff Model and estimate the probability of the word "transforms" using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.

Answer

The trigram machine learning transforms does not occur in the corpus. Therefore, the model backs off to P(transforms | learning). The bigram learning transforms also does not occur. Finally, the model checks the unigram P(transforms). The word "transforms" does not occur in the corpus, so its unigram probability is 0. Hence the final backoff probability is 0 for this small corpus. Backoff is useful because real-world search queries contain many combinations that were not present in the training corpus. In a larger corpus, a lower-order model may still provide a non-zero probability instead of immediately failing when a trigram is unseen.

Question 3

Using Deleted Interpolation, calculate the probability of the sequence “machine learning improves” using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.

Answer

The interpolation formula is P(w|w■,w■) = λ■P(w|w■,w■) + λ■P(w|w■) + λ■P(w).

Given λ■

= 0.5, λ■ = 0.3, and λ■ = 0.2. The trigram probability is P(improves | machine, learning) = 1/3 = 0.3333. The bigram probability is P(improves | learning) = 1/3 = 0.3333. The corpus contains 15 words and “improves” occurs once, so P(improves) = 1/15 = 0.0667. Therefore, P = 0.5(0.3333) + 0.3(0.3333) + 0.2(0.0667) = 0.27998, approximately 0.28. Interpolation is more robust because it does not depend completely on one N-gram order; lower-order probabilities contribute when higher-order statistics are sparse.

Question 4

Calculate the Entropy of the prediction distribution P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

Answer

Entropy is H = −Σ p(x)log■p(x). Using the rounded probabilities gives approximately 1.581 bits. Since the three candidates are intended to be equally probable, the exact equal distribution is 1/3 for each word and H = log■(3) ≈ 1.585 bits. The relatively high entropy occurs because the three candidate words have almost equal probabilities. The system therefore has low confidence in selecting one specific next word. Higher entropy indicates higher uncertainty and lower prediction confidence.

Question 5

Develop a Python program using NLTK for N-gram generation, NumPy for probability computations, and the Math library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

Answer – Python Program

```
import nltk import
numpy as np import
math
from collections import Counter
nltk.download("punkt") corpus = """
machine learning improves business
machine learning enables automation
machine learning drives innovation
""" tokens =
nltk.word_tokenize(corpus.lower())
unigram = Counter(tokens) bigram = Counter(zip(tokens,
tokens[1:])) trigram = Counter(zip(tokens, tokens[1:],
tokens[2:])) print("=" * 60) print("E-COMMERCE SMART
SEARCH AND PRODUCT PREDICTION")
print("=" * 60)

print("\nUNIGRAM COUNTS") print(unigram)

print("\nBIGRAM COUNTS") print(bigram)

print("\nTRIGRAM COUNTS") print(trigram)

bigram_probability = ( bigram[("machine",

"learning")] / unigram["machine"]
) print("\n1. MLE BIGRAM
PROBABILITY")
print("P(learning | machine) =", bigram_probability)

print("\n2. BACKOFF MODEL") trigram_p
= ( trigram[("machine", "learning",
"transforms")] /
```

```
bigram[("machine", "learning")] if
bigram[("machine", "learning")] > 0 else 0 )
bigram_p = ( bigram[("learning",
"transforms")] /
unigram["learning"] if unigram["learning"] > 0
else 0 ) unigram_p = unigram["transforms"] /
len(tokens)
print("Trigram probability:", trigram_p)
print("Bigram probability:", bigram_p) print("Unigram
probability:", unigram_p)
backoff_probability = (
trigram_p if trigram_p > 0 else bigram_p if
bigram_p > 0 else unigram_p ) print("Backoff
probability:", backoff_probability)

print("\n3. DELETED INTERPOLATION")

lambda1 = 0.5 lambda2
= 0.3 lambda3 = 0.2

tri = trigram[("machine", "learning", "improves")] / bigram[("machine", "learning")]
bi = bigram[("learning", "improves")] / unigram["learning"] uni = unigram["improves"]

/ len(tokens) interpolated = lambda1 * tri + lambda2 * bi +

lambda3 * uni

print("Trigram probability:", tri)
print("Bigram probability:", bi) print("Unigram
probability:", uni)
print("Final interpolation probability:", interpolated)

print("\n4. ENTROPY") probabilities = np.array([0.33,

0.33, 0.33]) entropy = -

sum(p * math.log2(p) for p in probabilities)

print("Entropy:", entropy, "bits")

print("\nFINAL NEXT-WORD PREDICTIONS") print("improves
: 0.33") print("enables : 0.33") print("drives : 0.33")
```

CASE STUDY 2: AI-POWERED HOSPITAL APPOINTMENT CHATBOT

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs Part-of-Speech tagging before identifying user intentions and generating responses.

Consider the following user inputs:

1. “Book an appointment with the doctor.”

2. “The book contains medical information.”

The chatbot uses the Penn Treebank POS Tagset and an HMM-based probabilistic POS tagger to determine the grammatical role of ambiguous words.

For the word “book”, the system is given: P(book | VB) = 0.7, P(book | NN) = 0.3, P(Start → VB) = 0.6, and P(Start → NN) = 0.4. The system must determine whether “book” functions as a Verb (VB) or Noun (NN) depending on its context.

Question 1

Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the POS tag assigned to “book” using the grammatical context of each sentence.

Answer

Sentence 1: “Book an appointment with the doctor.”

Word	POS Tag	Meaning
Book	VB	Verb
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun

.

Sentence 2: “The book contains medical information.”

Word	POS Tag	Meaning
The	DT	Determiner
book	NN	Noun
Word	**POS Tag**	**Meaning**
contains	VBZ	Verb
medical	JJ	Adjective
information	NN	Noun

Tagged sequence: The/DT book/NN contains/VBZ medical/JJ information/NN. Here “book” is a noun because it refers to an object. Thus the same word can have different grammatical roles depending on context.

Question 2

Using the given HMM probabilities, compute the likelihood of “book” being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.

Answer

For VB: P(VB, book) = P(VB | Start) × P(book | VB) = 0.6 × 0.7 = 0.42. For NN: P(NN, book) = P(NN | Start) × P(book | NN) = 0.4 × 0.3 = 0.12. Since 0.42 > 0.12, the given HMM favors VB at the beginning of a sentence. In an actual complete sentence, an HMM also considers surrounding tag transitions, so context such as “The book” can strongly favor NN.

Question 3

Compare Rule-Based POS Tagging and Stochastic HMM Tagging for resolving the ambiguity of the word “book”. Discuss the advantages and limitations of both approaches in a healthcare chatbot.

Answer

Feature	Rule-Based	Stochastic HMM
Approach	Hand-written rules	Probabilities
Training data	Not necessarily required	Required
Context	Rule-defined	Statistical context
Ambiguity	Depends on rules	Probability-based
Flexibility	Lower	Higher
Explainability	High	Moderate
Maintenance	Rules must be updated	Retraining may be required

Rule-based tagging is simple and explainable, but it can become difficult to maintain and may not handle every ambiguous context. HMM tagging uses language statistics and can handle ambiguity probabilistically, but it requires suitable training data and may suffer from unseen words or sequences. In a healthcare chatbot, both methods can support reliable interpretation of commands such as “Book an appointment.”

Question 4

Evaluate how standardized POS tagsets such as the Penn Treebank Tagset can support downstream NLP tasks such as intent detection, entity identification, and response generation in healthcare chatbots.

Answer

A standardized POS tagset provides consistent grammatical information to downstream NLP modules. For intent detection, the chatbot can identify actions such as book, cancel, and reschedule. For entity identification, POS information can help identify important concepts such as doctor, appointment, hospital, medicine, and date. For response generation, grammatical information helps the system understand the structure of the user's request and generate an appropriate response. The overall pipeline can be represented as POS tagging → syntactic understanding → intent/entity extraction → response generation.

Question 5

Develop a Python program using NLTK to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple HMM-based POS tagging calculation using the given emission and transition probabilities. The program should calculate the probability of “book” being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

```
import nltk

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger_eng")

sentences = [ "Book an appointment with
the doctor.", "The book contains medical
information." ] print("=" * 60) print("AI-
POWERED HOSPITAL APPOINTMENT CHATBOT")

print("=" * 60)

for sentence in sentences: tokens =
nltk.word_tokenize(sentence) tags = nltk.pos_tag(tokens)

print("\nSentence:") print(sentence) print("\nTokens:")

print(tokens)

print("\nPenn Treebank POS Tags:") print(tags)

p_book_vb = 0.7 p_book_nn
= 0.3 p_start_vb = 0.6
p_start_nn = 0.4

prob_vb = p_start_vb * p_book_vb prob_nn

= p_start_nn * p_book_nn

print("\nHMM PROBABILITY CALCULATION")
print("P(VB | Start) =", p_start_vb) print("P(book
| VB) =", p_book_vb) print("P(VB, book) =",
prob_vb)

print("\nP(NN | Start) =", p_start_nn)
print("P(book | NN) =", p_book_nn) print("P(NN,

book) =", prob_nn) best_tag = "VB" if prob_vb >

prob_nn else "NN"

print("\nMost Probable Tag for 'book':", best_tag)
```

CASE STUDY 3: FINANCIAL NEWS POS TAG CORRECTION AND UNCERTAINTY ANALYSIS

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a Transformation-Based Tagger (Brill Tagger) to correct common tagging errors.

The sentence processed by the system is: “Market growth drives investment.”

The initial POS tags produced by the system are: market/NN, growth/NN, drives/NNS, investment/NN.

A learned transformation rule states: Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies: market – 500 occurrences; growth – 350 occurrences; drives – 180 occurrences; investment – 420 occurrences.

Question 1

Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POStag sequence and explain why the transformation is linguistically appropriate.

Answer

Initial sequence: market/NN, growth/NN, drives/NNS, investment/NN. The word “drives” is tagged NNS and its preceding word “growth” is tagged NN. Therefore, the rule applies and changes drives/NNS to drives/VBZ. Corrected sequence: market/NN, growth/NN, drives/VBZ, investment/NN. The transformation is linguistically appropriate because “drives” is the action performed by the subject “market growth”, so it is a third-person singular present-tense verb.

Question 2

Analyze the initial tagging error for “drives”. Explain why the word should receive the VBZ tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.

Answer

In “Market growth drives investment”, the noun phrase “market growth” functions as the subject, “drives” expresses the action, and “investment” functions as the object. Therefore, “drives” is a third-person singular present-tense verb and should receive the Penn Treebank tag VBZ. The initial NNS tag is incorrect because NNS represents a plural noun. The surrounding grammatical context identifies the noun phrase followed by a predicate verb, making drives/VBZ linguistically appropriate.

Question 3

Using the given corpus statistics, calculate the relative frequency/probability of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.

Answer

The total frequency is 500 + 350 + 180 + 420 = 1450. Relative probability is frequency divided by total frequency.

Word	Frequency	Relative Probability
market	500	500/1450 = 0.3448
growth	350	350/1450 = 0.2414
drives	180	180/1450 = 0.1241
investment	420	420/1450 = 0.2897
Total	1450	1.0000

Corpus frequency provides statistical information about how commonly words occur. It can support language modeling, POS tagging, word prediction, text classification, speech recognition, and information retrieval. Frequency information can influence the statistical confidence assigned to candidate words or tags.

Question 4

Calculate the entropy of the word-frequency distribution before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.

Answer

Entropy is H = −ΣP(w)log■P(w). Using the probabilities 0.3448, 0.2414, 0.1241, and 0.2897 gives H ≈ 1.9161 bits. The transformation changes drives/NNS to drives/VBZ but does not change the word frequencies. Therefore, the word-frequency distribution remains unchanged and the entropy remains the same: H before = 1.9161 bits and H after = 1.9161 bits. This shows that transformation-based correction improves grammatical tagging without changing the underlying word-frequency distribution. Entropy measures uncertainty in the distribution; it does not directly measure whether the POS tag is correct unless a tag-probability distribution is being evaluated.

Question 5

Develop a Python program using NLTK for initial POS tagging and implement a simple Transformation-Based Tagger that applies the rule “Change NNS to VBZ if the preceding word is tagged as NN.” The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python math library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Answer – Python Program

```
import nltk

import math


nltk.download("punkt")
nltk.download("averaged_perceptron_tagger_eng")

sentence = "Market growth drives investment." tokens

= nltk.word_tokenize(sentence)

initial_tags = [

("market", "NN"),

("growth", "NN"),
("drives", "NNS"),

("investment", "NN") ]
```

```
print("=" * 60) print("FINANCIAL NEWS POS TAG
CORRECTION") print("="
* 60) print("\nTokens:")
print(tokens)
print("\nInitial POS Tags:") for
word, tag in initial_tags:
print(word + "/" + tag)
corrected_tags = initial_tags.copy()
for i in range(1, len(corrected_tags)): current_word,
current_tag = corrected_tags[i] previous_word,
previous_tag = corrected_tags[i - 1] if current_tag
== "NNS" and previous_tag == "NN":
corrected_tags[i] = (current_word, "VBZ")
print("\nCorrected POS Tags:") for
word, tag in corrected_tags:
print(word + "/" + tag)
frequencies = { "market":
500,
"growth": 350,
"drives": 180,
"investment": 420
} total =
sum(frequencies.values())
probabilities = {}
print("\nCORPUS FREQUENCY ANALYSIS") for
word, frequency in frequencies.items():
probability = frequency / total probabilities[word]
= probability
print( f"{word:12} " f"Frequency =
{frequency:4} " f"Probability =
{probability:.4f}" ) entropy_before
= 0
for probability in probabilities.values():
entropy_before -= probability * math.log2(probability)
entropy_after = entropy_before
print("\nENTROPY ANALYSIS") print("Entropy before
transformation:", round(entropy_before, 4),
"bits") print("Entropy after transformation:",
round(entropy_after, 4), "bits")
```

```
print("\nFINAL POS-TAGGING RESULT:") for
word, tag in corrected_tags:
print(f"{word}/{tag}")
```